
Documentação de Projeto

para o sistema

< SISTEMA DRIVEFLOW >

Versão 1.0

Projeto de sistema elaborado pelo aluno João Pedro Moura Santos
como parte da disciplina **Projeto de Software**.

<05/06/2026>

Sumário

1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso e Histórias de Usuários	1
2.3 Diagrama de Sequência do Sistema e Contrato de Operações	1
3. Modelos de Projeto	1
3.1 Arquitetura	1
3.2 Diagrama de Componentes e Implantação.	2
3.3 Diagrama de Classes	2
3.4 Diagramas de Sequência	2
3.5 Diagramas de Comunicação	2
3.6 Diagramas de Estados	2
4. Modelos de Dados	2

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
João Moura	05/06	Criação do projeto/diagramas	1.0
João moura	10/06	Finalizando projeto/ correções Estrutura diagramas	1.1

1. Introdução

Este documento apresenta o projeto de engenharia de software do **DriveFlow**, uma plataforma moderna de aluguel de carros. O sistema foi concebido utilizando uma **Arquitetura Monolítica estruturada em 3 Camadas (3-Tier Architecture)**, integrando a interface do usuário, a lógica de negócio e o armazenamento de dados em um ecossistema coeso e de fácil manutenção.

A camada de apresentação é implementada como uma *Single Page Application* (SPA) em **React**, que se comunica de forma síncrona e segura via requisições HTTPS/JSON com o servidor de aplicação backend, desenvolvido em **Java Spring Boot**. Toda a persistência e inteligência de dados são centralizadas em um único banco de dados relacional robusto utilizando o SGBD **PostgreSQL**.

2. Modelos de Usuário e Requisitos

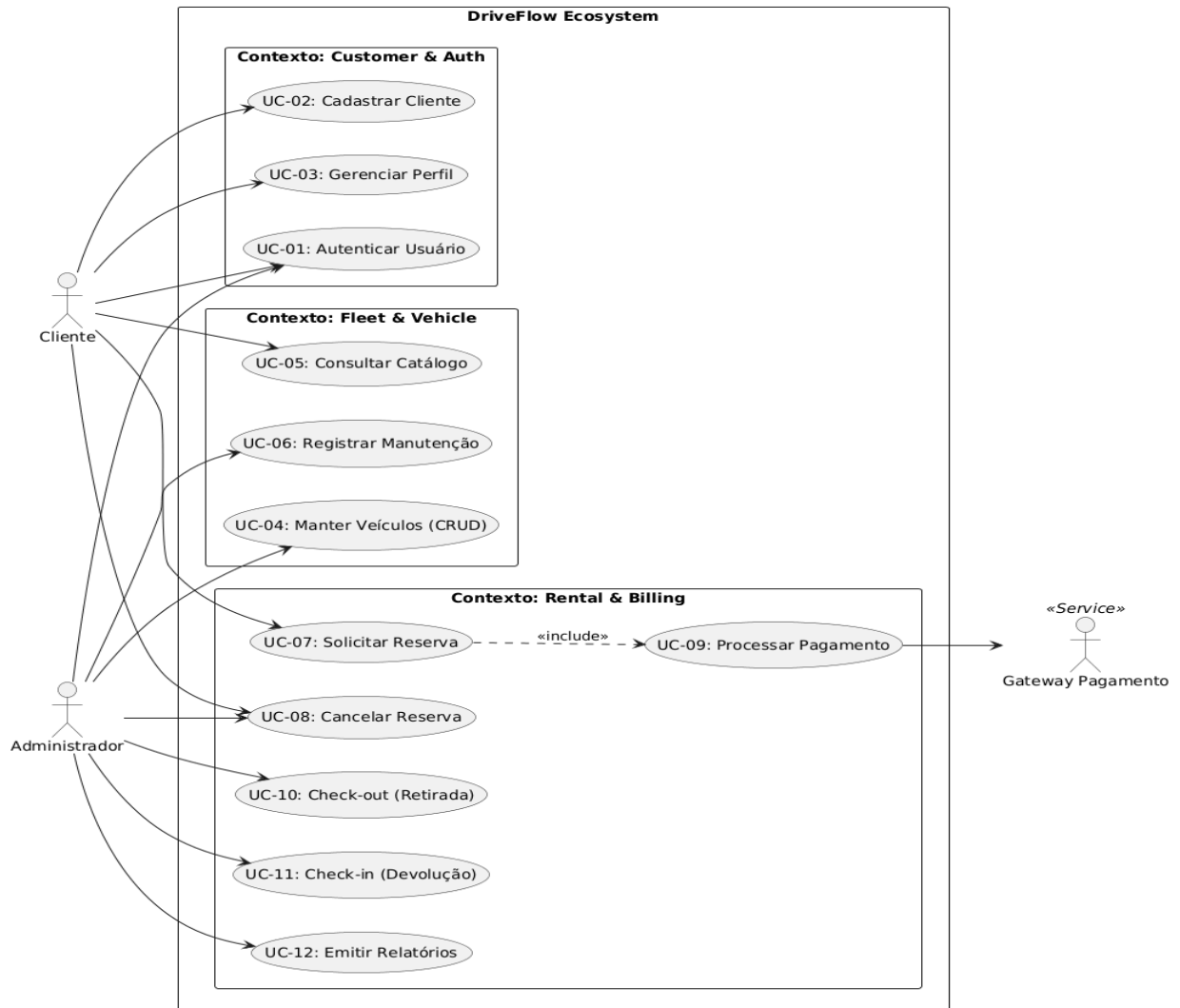
2.1 Descrição de Atores

- **Cliente:** Usuário final do sistema. Realiza cadastro, consulta o catálogo de automóveis em tempo real, solicita reservas, realiza pagamentos e gerencia seu perfil.
- **Administrador:** Funcionário ou gerente da locadora. Responsável pelo controle operacional, incluindo cadastro da frota, aprovação de cadastros, liberação e recebimento de veículos, e análise de faturamento.
- **Gateway de Pagamento (Ator Externo):** Sistema parceiro responsável por processar as transações de cartão de crédito e PIX de forma segura.

2.2 Modelo de Casos de Uso

- **UC-01: Autenticar Usuário:** Permite que clientes e administradores realizem login, logout e validação de tokens JWT através do Gateway de Segurança.
- **UC-02: Cadastrar Cliente:** Permite que novos clientes criem contas informando dados pessoais, e-mail, senha e número da CNH.
- **UC-03: Gerenciar Perfil do Usuário:** Permite ao cliente atualizar seus dados cadastrais, informações de contato e alterar credenciais de acesso.
- **UC-04: Manter Veículos (CRUD):** Permite ao Administrador cadastrar, listar, atualizar e remover veículos da frota da locadora.

- **UC-05: Consultar Catálogo de Veículos:** Permite ao Cliente buscar carros disponíveis aplicando filtros por marca, modelo, categoria (Econômico, SUV, Luxo) e faixa de preço.
- **UC-06: Registrar Manutenção de Veículo:** Permite ao Administrador alterar o estado de um carro para manutenção, bloqueando novas reservas para o período.
- **UC-07: Solicitar Reserva de Veículo:** Fluxo principal onde o Cliente seleciona o carro, define o período de locação e abre um pedido de reserva pendente.
- **UC-08: Cancelar Reserva:** Permite ao Cliente ou ao Administrador cancelar uma reserva ativa, aplicando regras de negócio sobre taxas e estornos.
- **UC-09: Processar Pagamento da Reserva:** Sistema consome o ator externo para aprovar ou recusar a transação financeira vinculada à reserva.
- **UC-10: Realizar Check-out do Veículo (Retirada):** Permite ao Administrador vistoriar o carro e registrar a entrega física e a chave para o cliente, ativando o contrato.
- **UC-11: Realizar Check-in do Veículo (Devolução):** Permite ao Administrador inspecionar o carro no retorno, calcular quilometragem, avarias e processar cobranças adicionais ou encerramento.
- **UC-12: Emitir Relatório de Faturamento e Ocupação:** Permite ao Administrador extrair métricas consolidadas sobre taxas de uso da frota e rendimentos mensais.



2.3 Diagrama de Sequência do Sistema

Diagrama de Sequência do Caso de uso 07 – Solicitar Reserva de Veículo

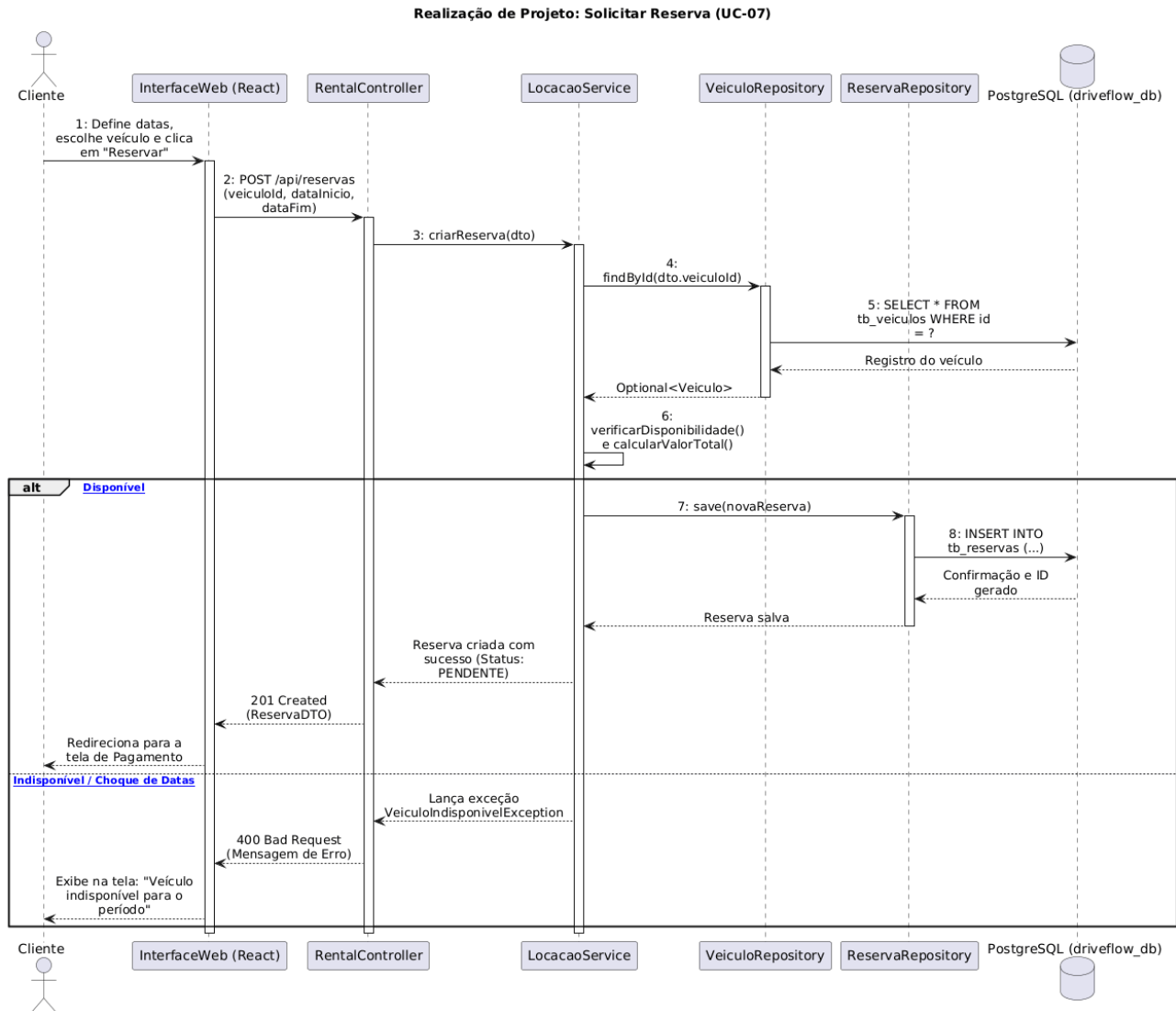


Diagrama de Sequência do Caso de Uso 04 – Manter Veículos

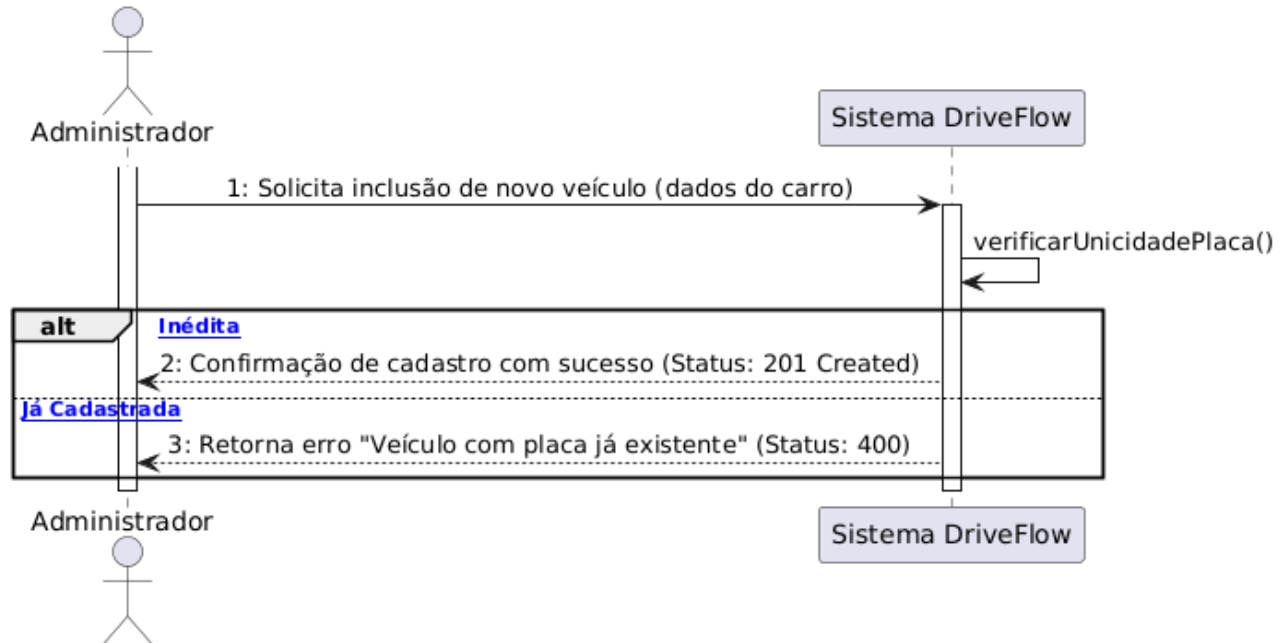
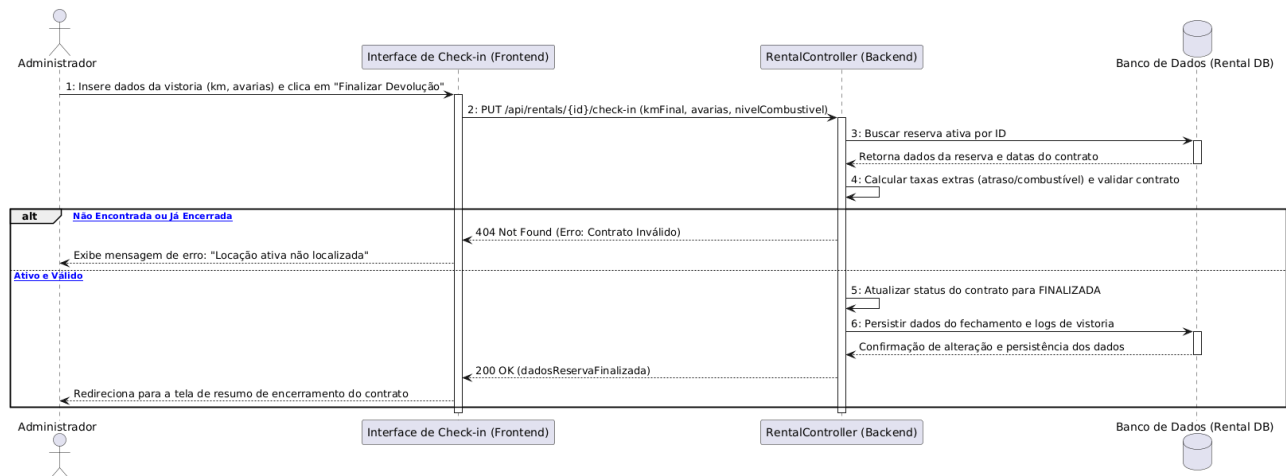
Diagrama de Sequência do Sistema: Manter Veículos (UC-04)

Diagrama de Sequência do Caso de Uso 11 – Realizar Check-In do Veículo



Contrato de Operação 1: UC-07 (Solicitar Reserva de Veículo)

Contrato	CO-01: Solicitação de Reserva de Automóvel
Operação	criarReserva(veiculold, dataInicio, dataFim)
Referências cruzadas	UC-07: Solicitar Reserva de Veículo e UC-05: Consultar Catálogo de Veículos.
Pré-condições	O cliente deve estar devidamente autenticado no ecossistema através de um token JWT válido. O veiculold informado deve corresponder a um veículo ativo cujo status seja DISPONIVEL durante todo o intervalo compreendido entre dataInicio e dataFim.
Pós-condições	Uma nova instância da classe Reserva foi criada no sistema (Status: PENDENTE). O preço da diária do veículo foi obtido localmente por meio de consulta direta ao VeiculoRepository. O valor total da reserva foi calculado programaticamente multiplicando o preço da diária pela quantidade de dias do intervalo solicitado. A nova instância foi associada ao Cliente solicitante e ao Veículo escolhido.

Contrato de Operação 2: UC-04 (Manter Veículos - Cadastrar Novo Veículo)

Contrato	CO-02: Cadastro de Veículo na Frota
Operação	cadastrarVeiculo(placa, modelo, marca, precoDiaria)
Referências cruzadas	UC-04: Manter Veículos (CRUD).
Pré-condições	O usuário requisitante deve estar autenticado e possuir o nível de acesso (Role) de Administrador. A placa enviada não pode existir previamente registrada no banco de dados (Vehicle DB).
Pós-condições	Uma nova instância v da classe Veiculo foi criada e persistida no banco de dados.v.placa recebeu o valor correspondente ao parâmetro placa.v.modelo, v.marca e v.precoDiaria foram inicializados com os respectivos dados fornecidos no formulário.v.status foi configurado por padrão com o estado inicial DISPONIVEL.

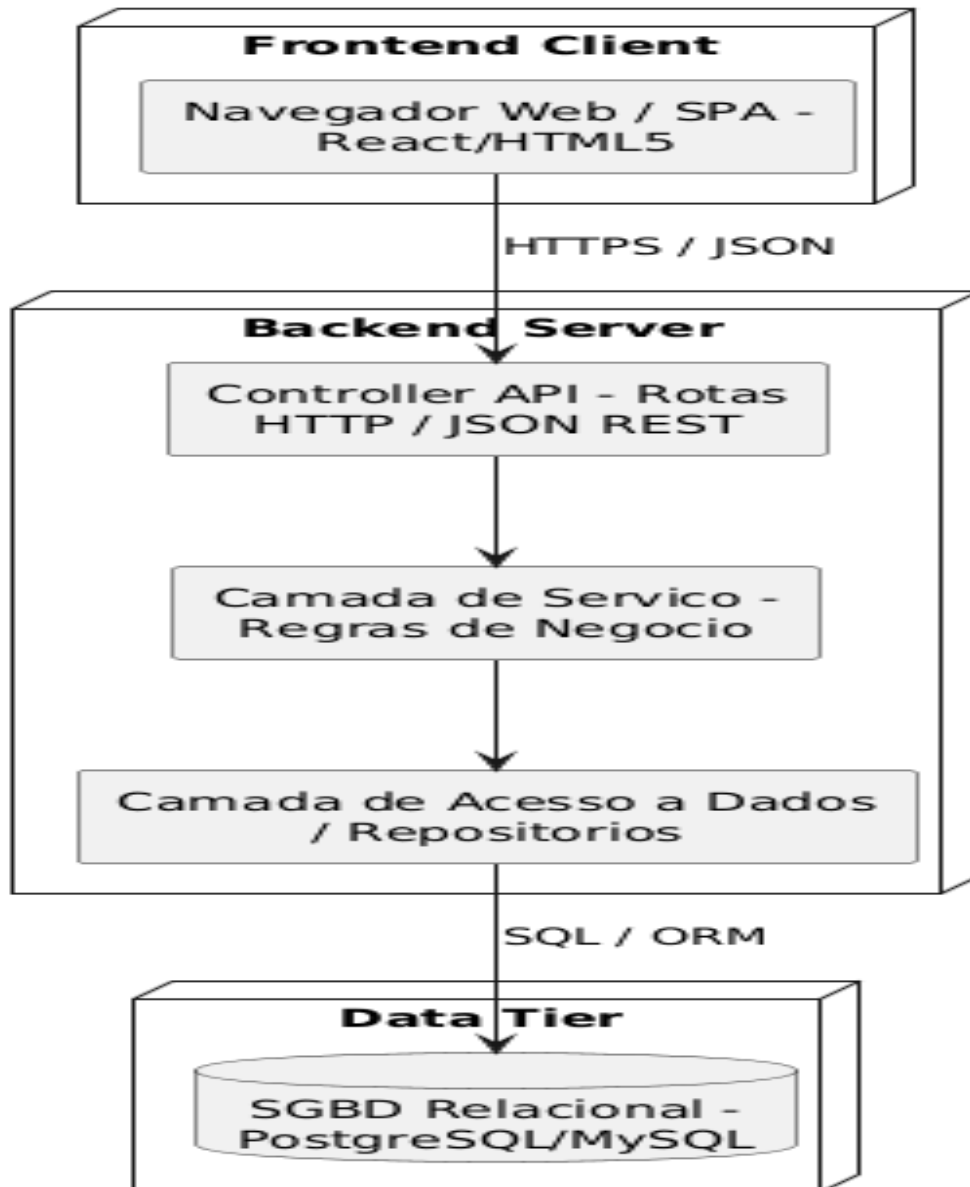
Contrato de Operação 3: UC-11 (Realizar Check-in do Veículo - Devolução)

Contrato	CO-03: Encerramento de Contrato e Check-in de Devolução
Operação	registrarCheckIn(reservald, kmFinal, avarias, nivelCombustivel)
Referências cruzadas	UC-11: Realizar Check-in do Veículo (Devolução).
Pré-condições	O usuário deve estar autenticado com credenciais de Administrador. A reserva identificada por reservald deve constar na base com o status atual igual a EM_ANDAMENTO.
Pós-condições	O status da instância r da classe Reserva identificada por reservald foi modificado para FINALIZADA. O veículo associado a r.veiculold teve seu status atualizado remotamente para DISPONIVEL (ou para MANUTENCAO, caso o parâmetro avarias registre danos estruturais que impeçam a locação imediata). O sistema calculou e registrou taxas adicionais baseadas em atrasos cronológicos ou divergências no nivelCombustivel, atualizando o fechamento financeiro do registro da locação.

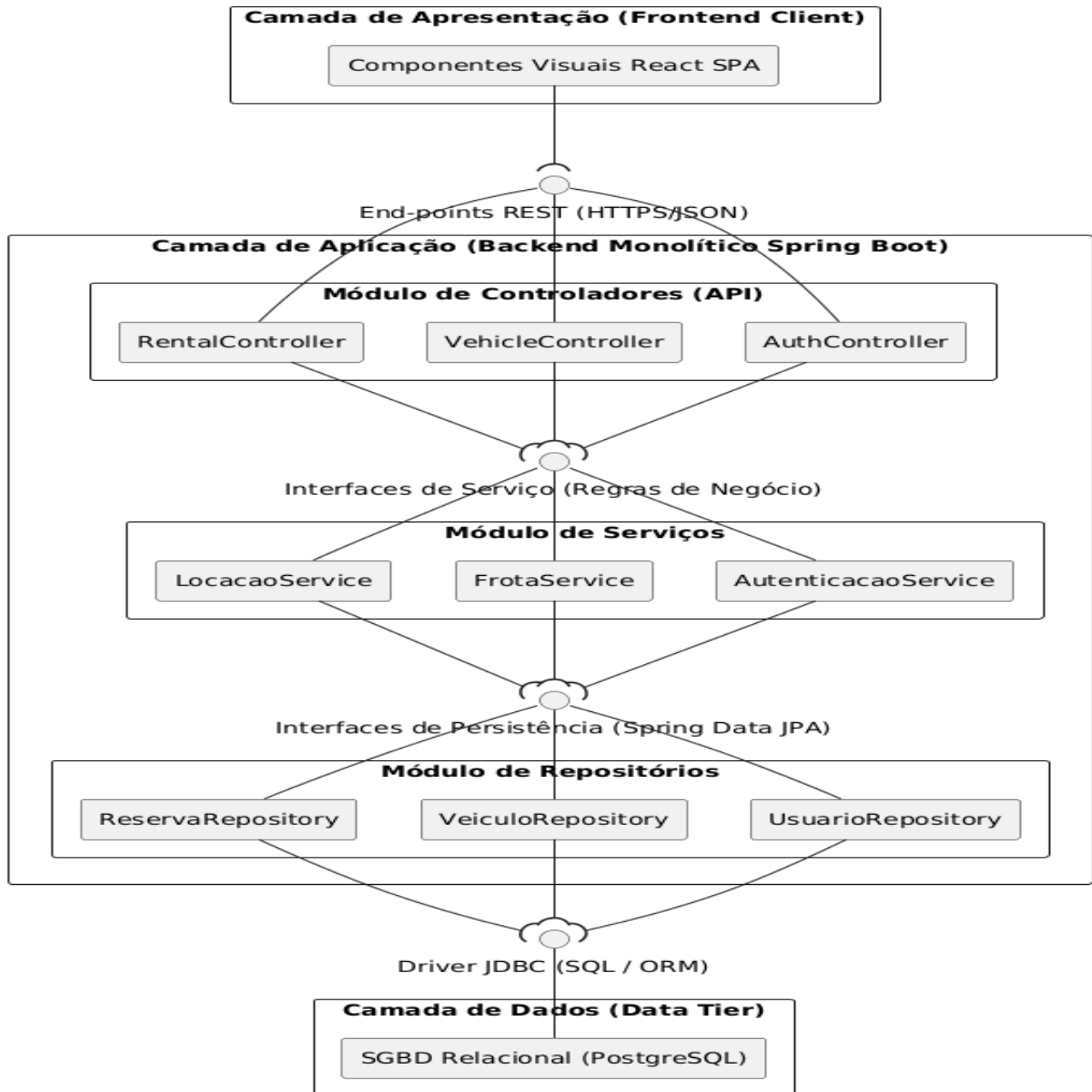
3. Modelos de Projeto

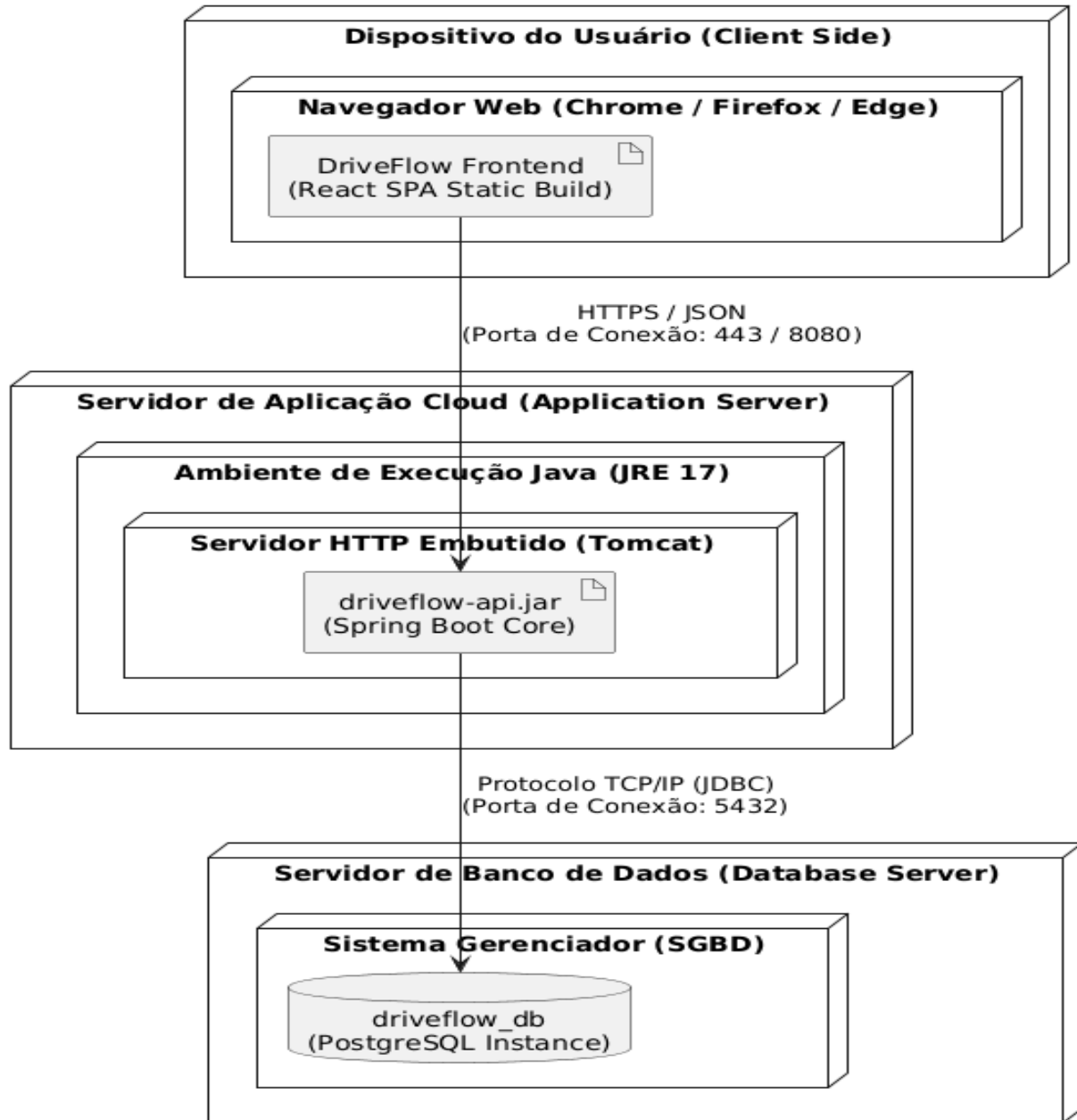
3.1 Arquitetura

Arquitetura em 3 Camadas - DriveFlow

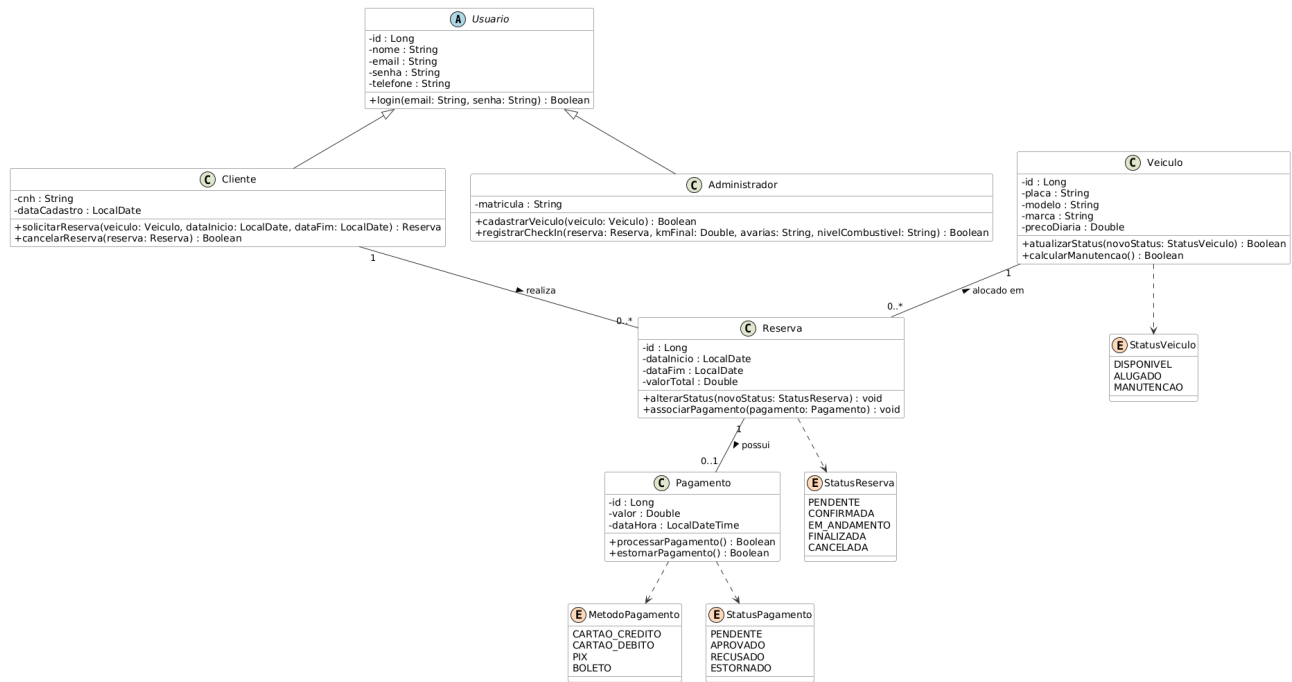


3.2 Diagrama de Componentes e Implantação.





3.3 Diagrama de Classes



3.4 Diagramas de Sequência

UC-07 - Solicitar Reserva de Veículo

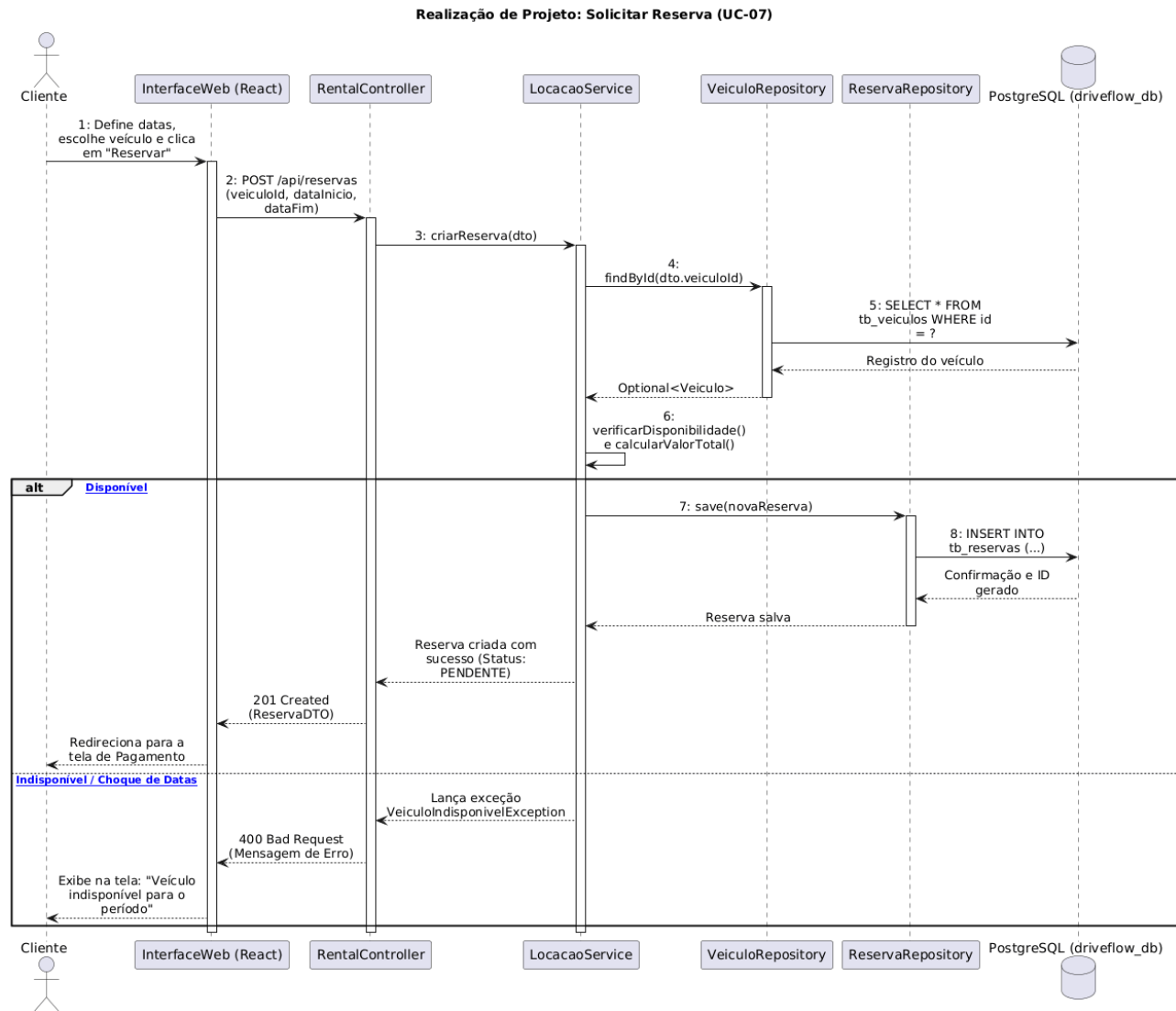
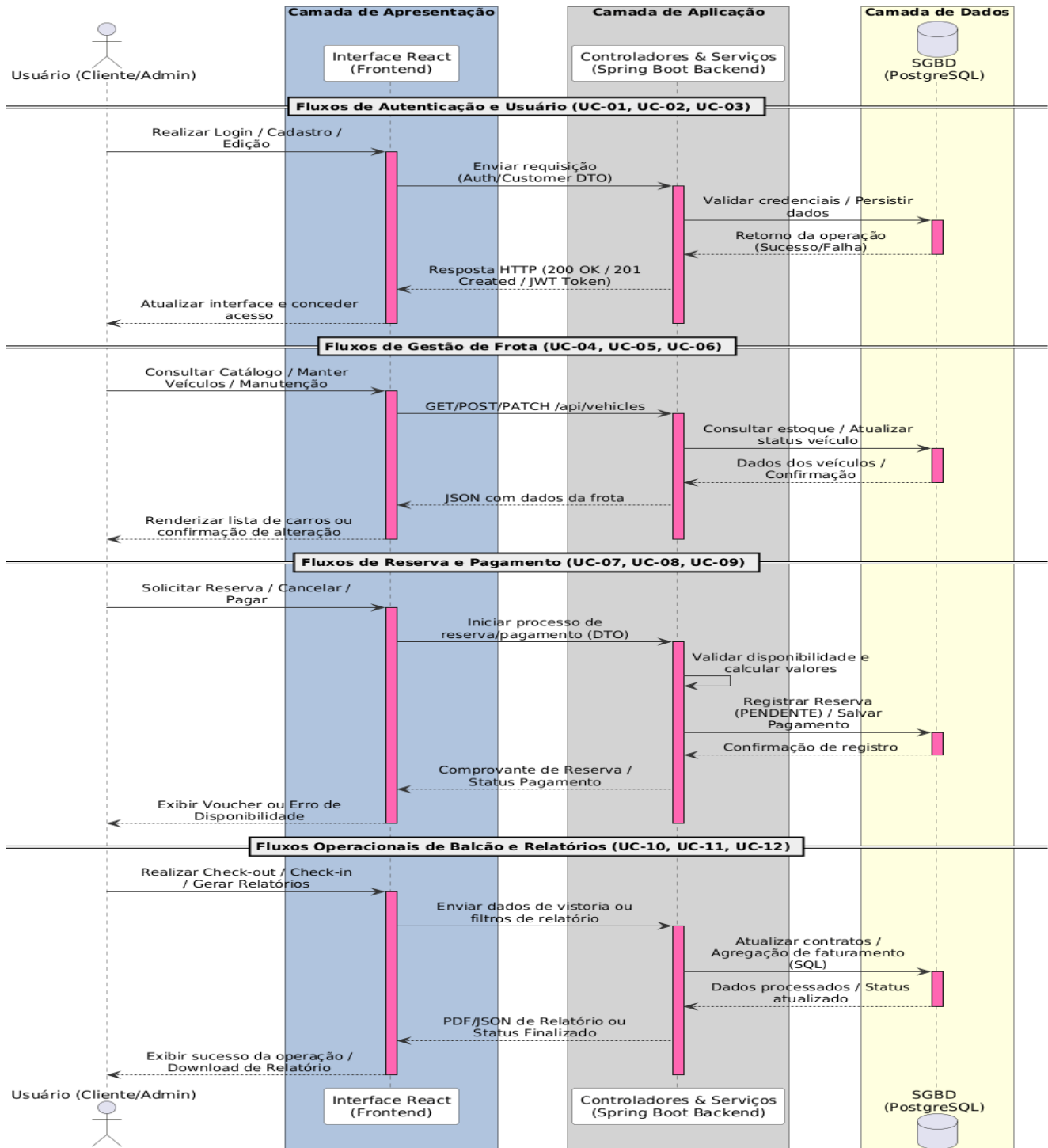
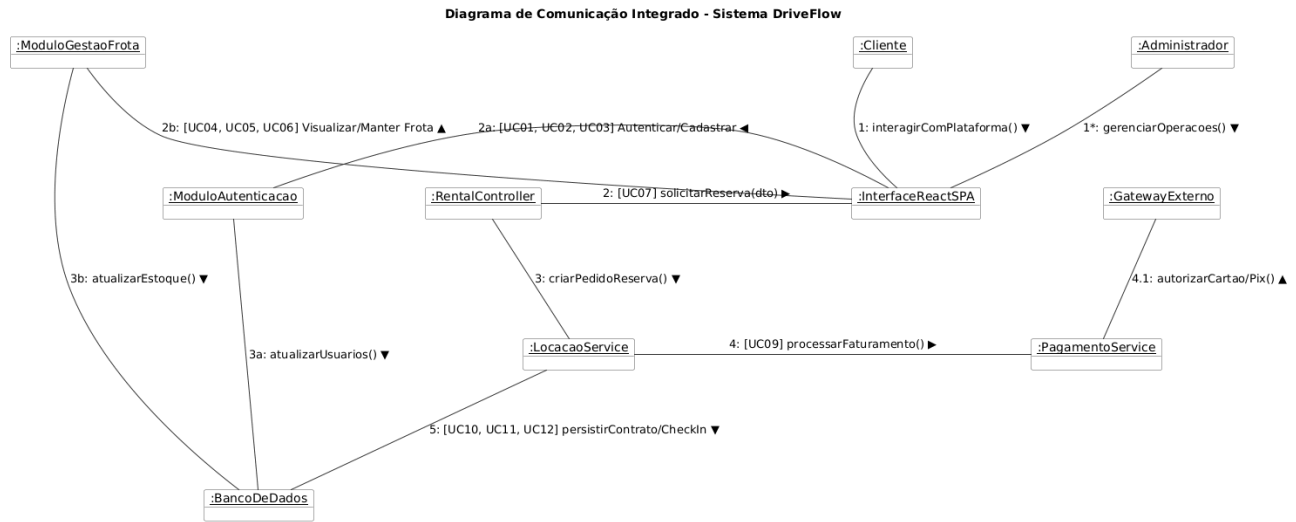


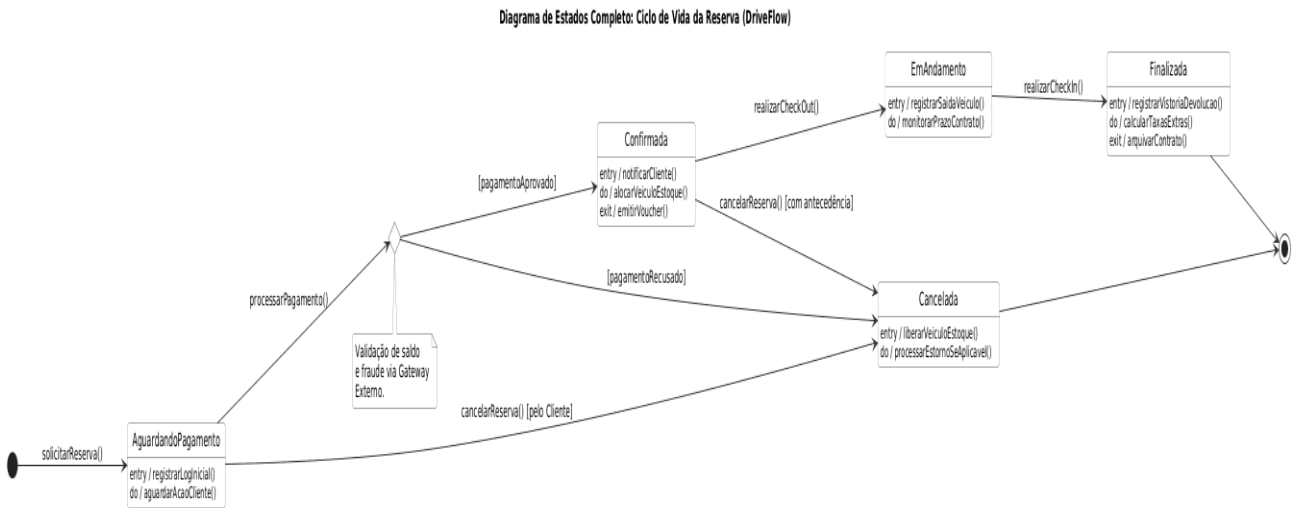
Diagrama de sequência - Geral



3.5 Diagramas de Comunicação



3.6 Diagramas de Estados



4. Modelos de Dados

4.1 Modelo Entidade-Relacionamento (ER) Conceitual

O modelo conceitual define as principais entidades do sistema e como elas interagem, independentemente da tecnologia de banco de dados:

- **Usuário:** Entidade generalizada que armazena os dados de autenticação (e-mail, senha). Especializa-se em **Cliente** (que possui CNH para alugar carros) e **Administrador** (que possui matrícula corporativa).
- **Veículo:** Representa os automóveis da frota. Contém identificadores únicos (placa), características físicas (marca, modelo) e o valor de precificação (preço da diária).
- **Reserva:** Entidade associativa fundamental. Ela materializa a relação de negócio entre um *Cliente* e um *Veículo* em um determinado intervalo de tempo (data início e data fim), totalizando os custos operacionais.
- **Pagamento:** Entidade financeira vinculada a uma reserva concluída, registrando a transação, método (Pix, Cartão) e autorização sistêmica.

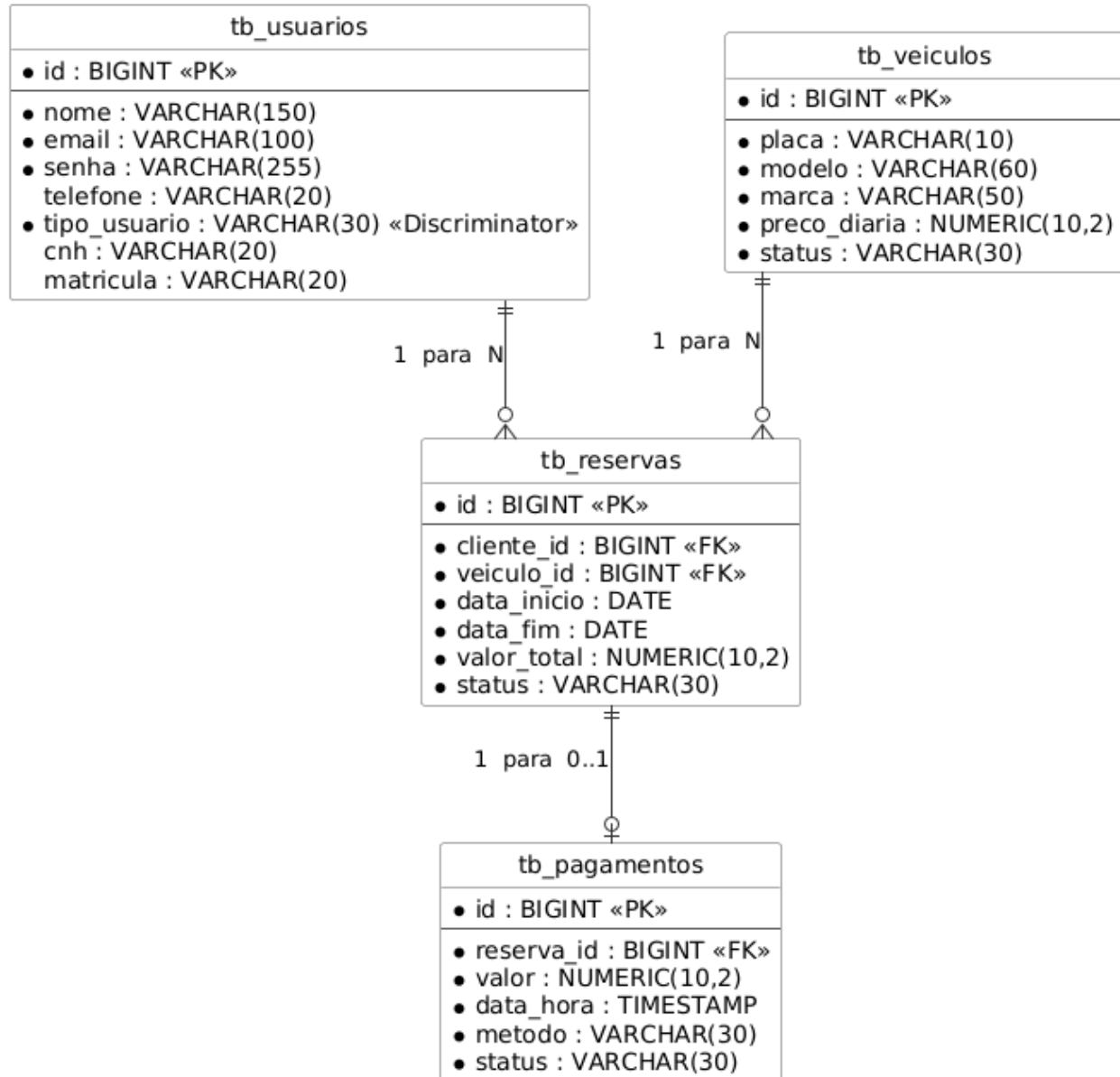
Regras de Relacionamento (Multiplicidade):

- Um Cliente pode realizar **zero ou muitas** Reservas (1:N).
- Um Veículo pode estar alocado em **zero ou muitas** Reservas em datas diferentes (1:N).
- Uma Reserva está vinculada a **exatamente um** Cliente e **exatamente um** Veículo.
- Uma Reserva gera **zero ou um** Pagamento (1:1), dependendo se o cliente já chegou na etapa de *checkout*.

4.2 Diagrama Entidade-Relacionamento (DER) Físico

O diagrama abaixo utiliza a notação de Engenharia da Informação (Pé de Galinha) para ilustrar o esquema físico do banco de dados, evidenciando as chaves primárias (PK) e chaves estrangeiras (FK). As linhas ortogonais garantem uma leitura limpa e sem cruzamentos sobrepostos.

Diagrama Entidade-Relacionamento Físico (DER) - DriveFlow



4.3 Estratégias de Mapeamento Objeto-Relacional (ORM)

Para traduzir a representação do paradigma orientado a objetos (classes em Java) para a representação não-objeto (tabelas e colunas relacionais do PostgreSQL), o sistema adota o framework **Spring Data JPA (Hibernate)**. As seguintes estratégias de mapeamento foram implementadas:

1. **Mapeamento de Identidade (Primary Keys):** Todas as entidades utilizam identificadores únicos substitutos (id numérico), mapeados com a anotação `@Id` e a estratégia `@GeneratedValue(strategy = GenerationType.IDENTITY)`. O banco de dados fica responsável por incrementar o sequencial de forma autônoma.
2. **Mapeamento de Herança (Estratégia *Single Table*):** Para evitar a complexidade estrutural de realizar *JOINS* toda vez que o usuário for autenticar no sistema, a herança da classe `Usuario` para `Cliente` e `Administrador` utiliza o mapeamento unificado em uma única tabela física (`tb_usuarios`). Foi utilizada a anotação `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)`. A coluna `tipo_usuario` atua como um `@DiscriminatorColumn`, indicando ao JPA qual subclasse instanciar ao resgatar um registro do banco.
3. **Mapeamento de Associações (Chaves Estrangeiras):**
 - a. **N:1 (Muitos para Um):** Nas classes de domínio, a entidade `Reserva` contém instâncias inteiras de `Cliente` e `Veiculo`. No mapeamento ORM, isso é traduzido utilizando a anotação `@ManyToOne` acoplada a um `@JoinColumn`. O Hibernate encarrega-se de gerar automaticamente as colunas `cliente_id` e `veiculo_id` como chaves estrangeiras com restrição de integridade no SGBD.
 - b. **1:1 (Um para Um):** A relação de exclusividade entre a `Reserva` e o `Pagamento` é mapeada através de `@OneToOne(cascade = CascadeType.ALL)`, garantindo que operações de exclusão ou atualização reflitam entre as entidades conectadas (integridade referencial).
4. **Tratamento de Dados Enumerados (Enums):** As regras de negócio dependem de status altamente controlados no Java (`StatusReserva`, `StatusVeiculo`, `MetodoPagamento`). Para garantir a legibilidade direta no banco de dados e facilitar consultas de suporte ou auditoria, os *Enums* são mapeados com `@Enumerated(EnumType.STRING)`. Isso converte o objeto de estado em texto simples (ex: "PENDENTE" ou "CONFIRMADA") na coluna SQL, em vez de salvar a posição numeral do Enum.